Date: 02-07-2026

Day 7 – CSS Display Properties and Flexbox

1. Inline Element

An inline element occupies only the width required by its content and does not start on a new line. Multiple inline elements can appear on the same line. These elements are mainly used to style or display small portions of text.

Examples: , <a>, <b>, <i>

2. Block Element

A block element occupies the entire width of its parent container and always starts on a new line. It is commonly used to divide a webpage into different sections.

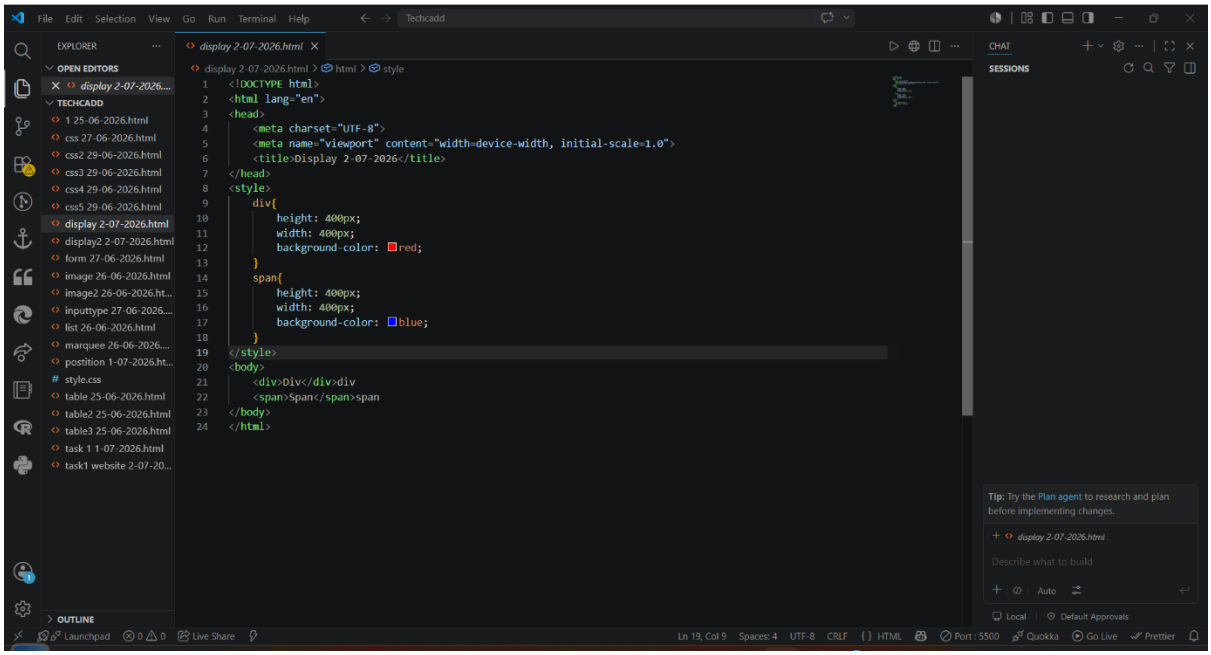
Examples: , <p>, <h1>, <section>

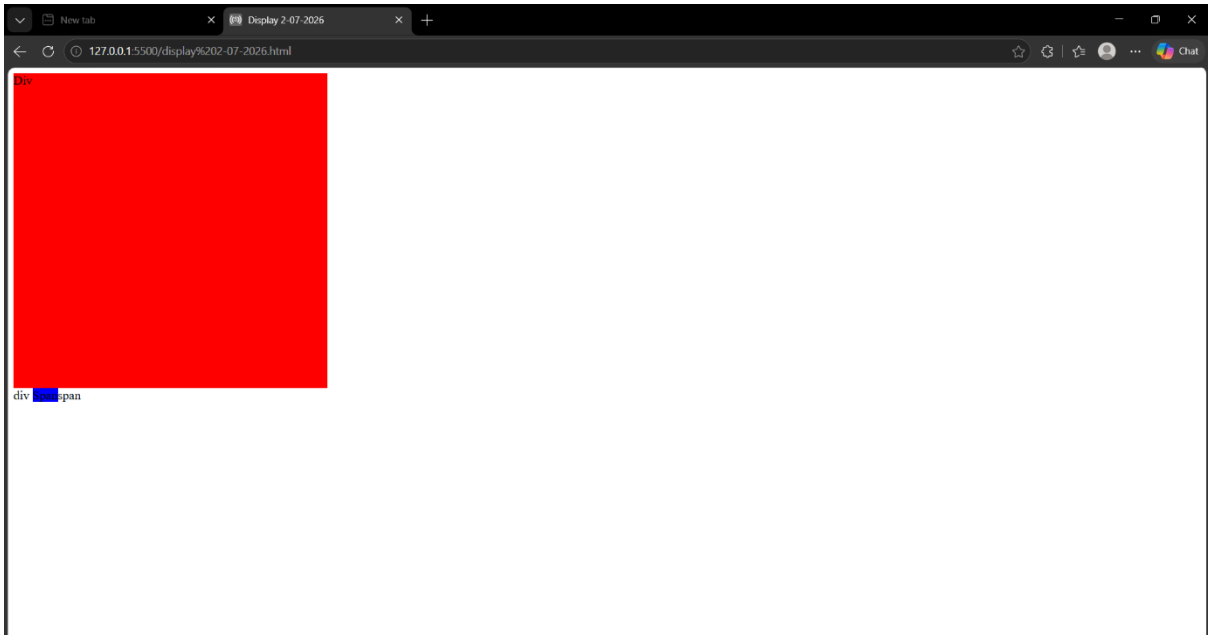
3. Display: Flex

The display: flex property converts a container into a flex container. It arranges all child elements in a flexible layout, making it easier to create responsive web pages.

Example:

```
.container{
    display: flex;
}
```





4. Flex Direction

The flex-direction property specifies the direction in which flex items are arranged.

Values	**Description**
row	Arranges items horizontally from left to right. It is the default value.
row-reverse	Arranges items horizontally from right to left.
column	Arranges items vertically from top to bottom.
column-reverse	Arranges items vertically from bottom to top.

Example:

```
.container{
    display: flex;
    flex-direction: row;
}
```

5. Gap

The gap property creates space between flex items without using margins. It helps keep the layout neat and organized.

Property	Description
gap	Adds equal spacing between flex items. The value can be in px, rem, %, etc.

Example:

```
.container{
    display: flex;
    gap: 20px;
}
```

6. Align Items

The align-items property aligns all flex items along the cross axis.

Values	Description
stretch	Stretches items to fill the container. It is the default value.
flex-start	Aligns items at the beginning of the cross axis.
center	Aligns items at the center of the cross axis.
flex-end	Aligns items at the end of the cross axis.
baseline	Aligns items according to the text baseline.

Example:

```
.container{
    display: flex;
    align-items: center;
}
```

7. Justify Content

The justify-content property aligns flex items along the main axis.

Values	Description
flex-start	Places items at the beginning of the container.
center	Places items at the center of the container.
flex-end	Places items at the end of the container.
space-between	Creates equal space between items with no space at the edges.
space-around	Creates equal space around every item.
space-evenly	Creates equal spacing between all items and the container edges.

Example:

```
.container{
    display: flex;
    justify-content: space-between;
}
```

8. Flex Wrap

The flex-wrap property determines whether flex items stay on one line or wrap onto multiple lines.

Values	Description
nowrap	All items remain on a single line. It is the default value.
wrap	Items move to the next line when there is not enough space.
wrap-reverse	Items wrap onto new lines in reverse order.

Example:

```
.container{
```

```
    display: flex;
    flex-wrap: wrap;
}
```

9. Align Self

The align-self property is used to align an individual flex item differently from the other items.

Values	Description
auto	Uses the parent's align-items value.
flex-start	Aligns the selected item at the beginning.
center	Aligns the selected item at the center.
flex-end	Aligns the selected item at the end.
stretch	Stretches the selected item to fill the available space.
baseline	Aligns the selected item according to the text baseline.

Example:

```
.item{
    align-self: flex-end;
}
```

10. Order

The order property changes the display order of flex items without changing the HTML code.

Values	Description
0	Default order of all flex items.
Positive Number	Items with smaller values appear before larger values.
Negative Number	Items with negative values appear before default values.

Example:

```
.item1{
    order: 2;
}
.item2{
    order: 1;
}
```

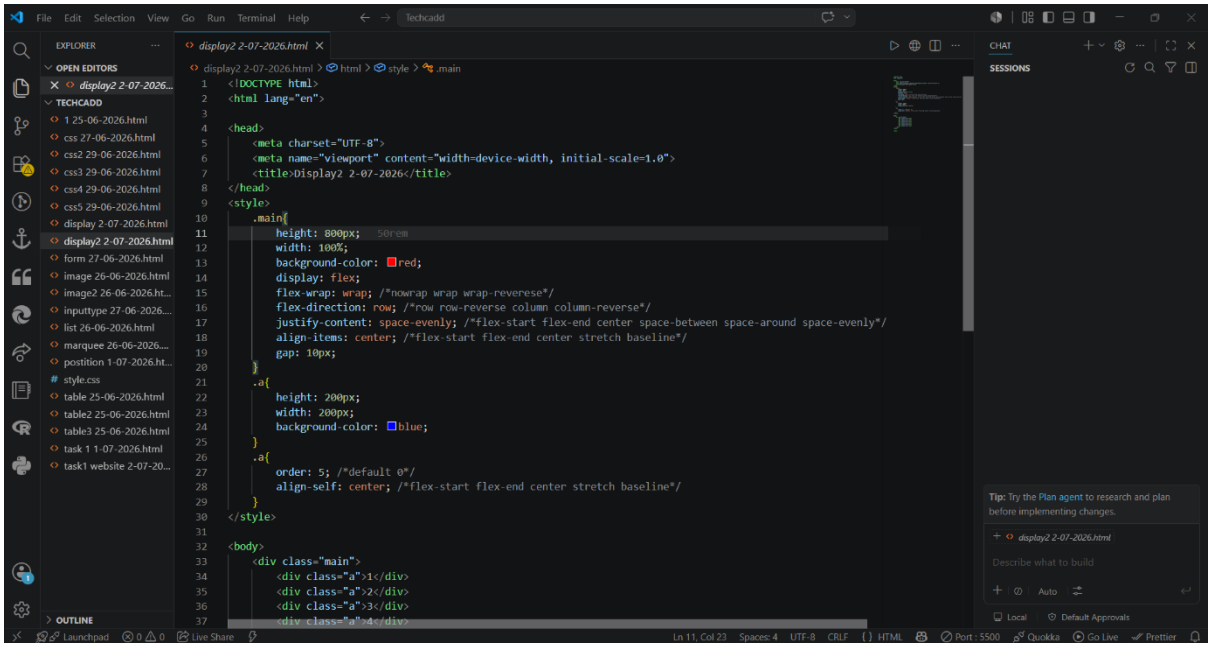
11. Span Tag ()

The tag is an inline HTML element used to apply CSS styles to a small portion of text without creating a new line.

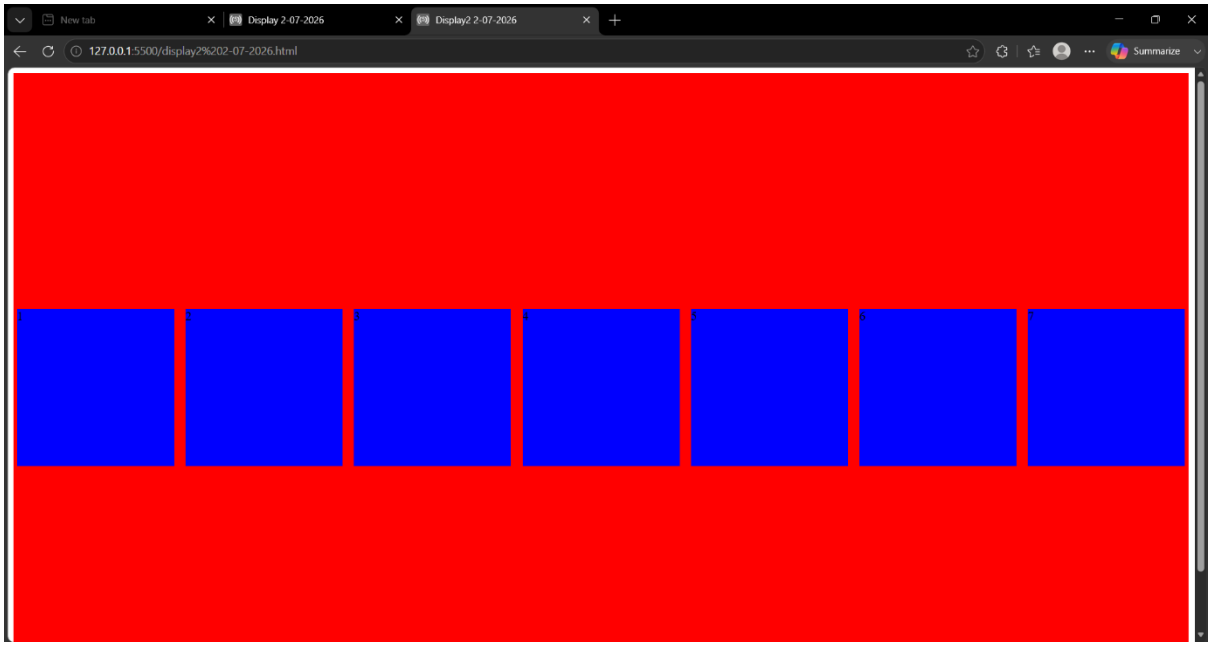
Example:

```
<p>Welcome to <span style="color:blue;">MERN Stack</span> Training.</p>
```

Code: -

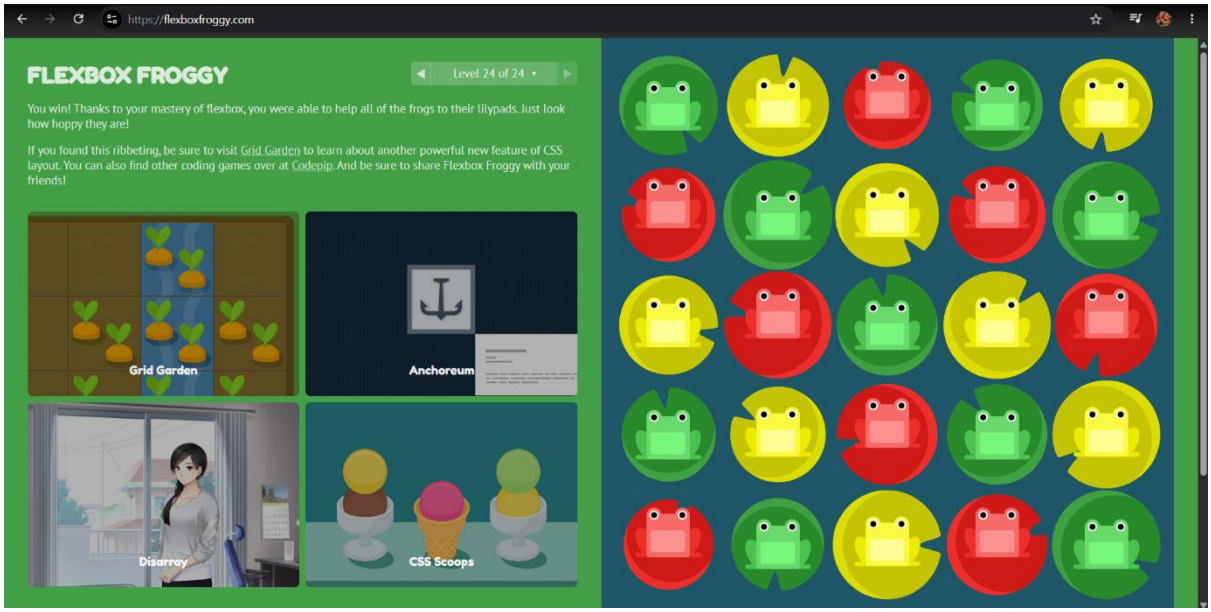


Date: 02-07-2026



Task 1 – Flexbox Froggy

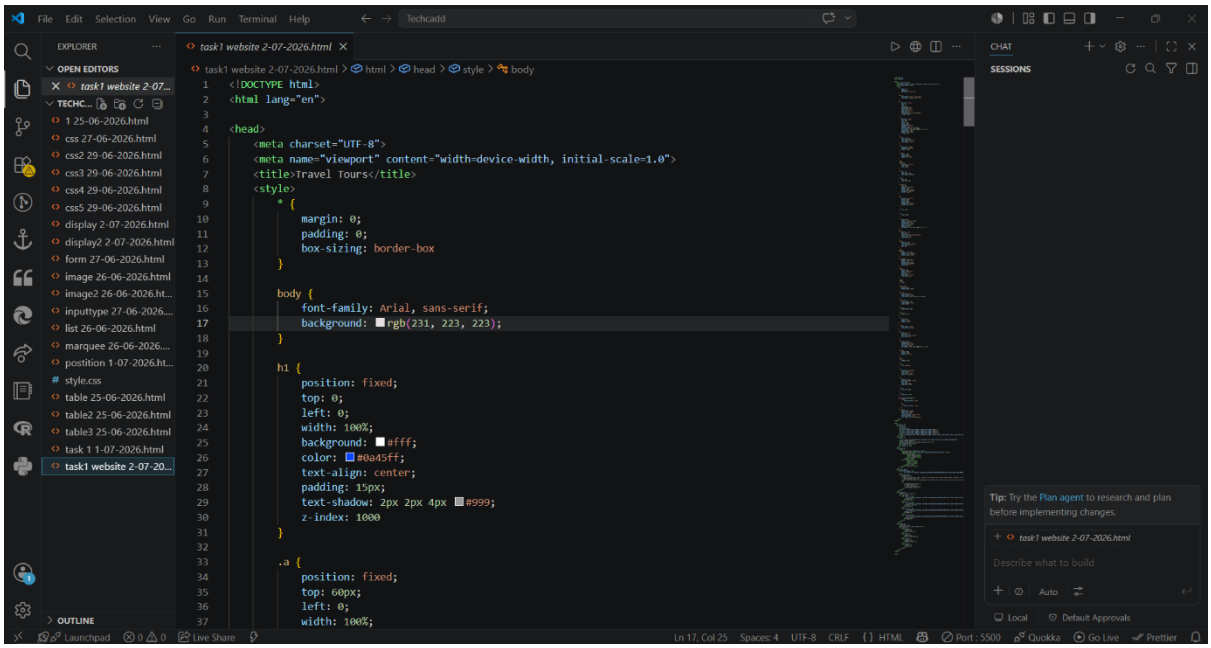
Today, I completed the Flexbox Froggy game to practice Flexbox concepts. I solved different levels by using properties such as justify-content, align-items, flex-direction, order, align-self, and flex-wrap. This activity helped me understand how Flexbox works in a fun and practical way.

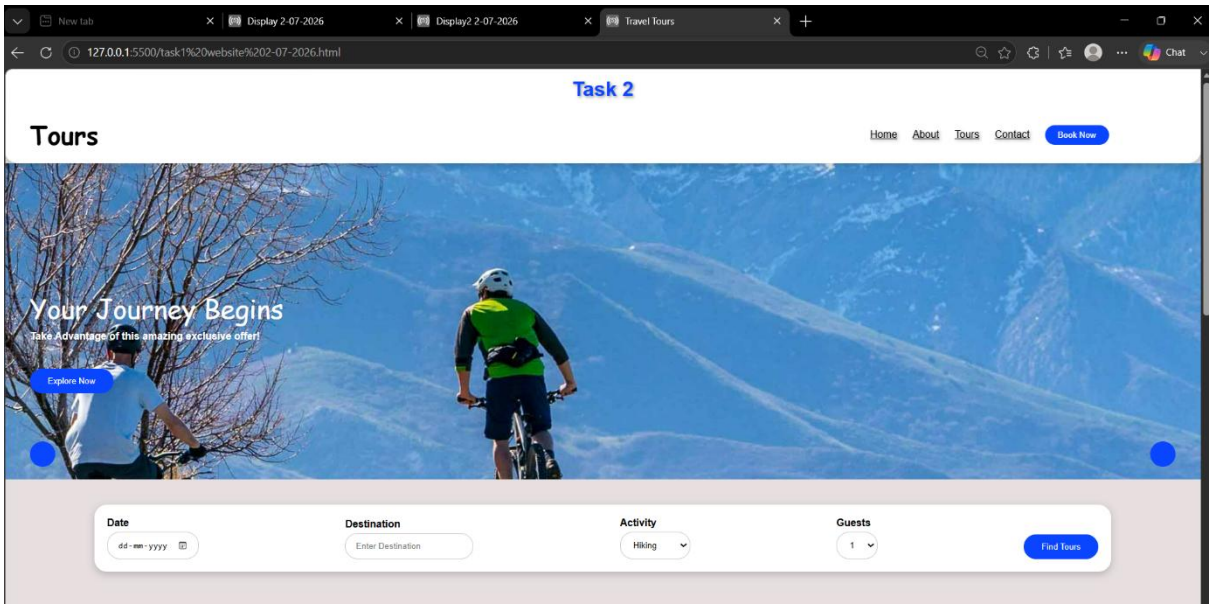


Task 2 – Simple Website Design

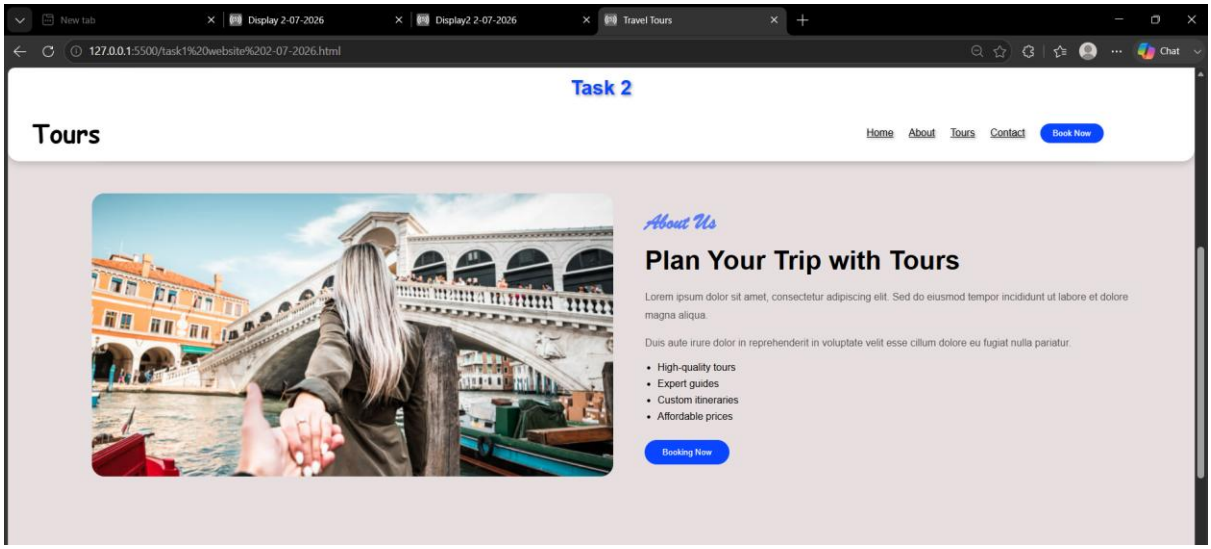
In this task, I created a simple website using HTML and CSS with a Header, Navigation Bar, Main Content Section, and Footer. I used Flexbox by applying

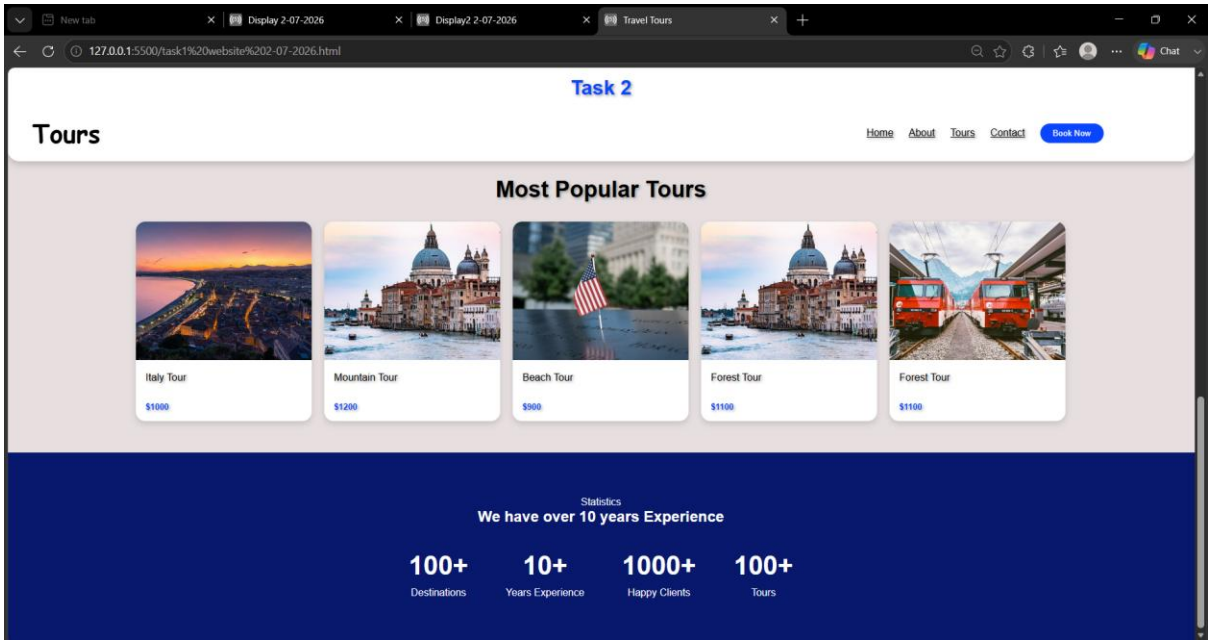
display: flex, flex-direction, justify-content, align-items, gap, flex-wrap, align-self, and order to arrange the webpage elements. I also used the tag to style specific text and applied colors, padding, margins, and borders to make the website responsive and visually attractive.





Date: 02-07-2026





Conclusion

Today, I learned the difference between inline and block elements and explored the CSS Flexbox layout. I practiced important Flexbox properties such as display: flex, flex-direction, gap, align-items, justify-content, flex-wrap, align-self, and order. I also learned the use of the tag for styling text. Finally, I completed the Flexbox Froggy game and designed a simple website using HTML and CSS, which improved my understanding of responsive webpage layouts.